

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 3
з дисципліни “Професійний Python”
на тему “Ітератори, генератори та потокова обробка даних”

Виконав
студент 3 курсу
групи ФЕП-32
Козачок Ростислав Михайлович

Перевірів
Куньо Іван Михайлович

Львів – 2026

Мета роботи: Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, `yield`, `generator expressions`, модулем `itertools`, принципом `lazy evaluation` та потоковою обробкою великих наборів даних.

Хід роботи:

Варіант 14. Потокова обробка системи керування завданнями Input.

- потокове читання та валідацію завдань (`task_id`, `title`, `description`, `priority`, `status`);
- фільтрацію за статусом та пріоритетом;
- лінійний пошук завдань та підрахунок виконаних/невиконаних;
- групування за статусом або пріоритетом після сортування;
- отримання перших N невиконаних завдань через `islice`;
- `batch processing` (пакетну обробку);
- лінійну трансформацію для формування короткого представлення;
- порівняння `eager` та `lazy` підходів (час і пам'ять) на наборах 10 000, 100 000 та 500 000 записів.

Опис структури даних:

Вхідні дані це згенеровані CSV-файли, що імітують вивантаження із системи управління завданнями.

Кожен запис містить 5 полів: `task_id` (int), `title` (str), `description` (str), `priority` (Low/Medium/High/Critical), `status` (Open/In Progress/Review/Done). Створено три файли: `tasks_10k.csv`, `tasks_100k.csv` та `tasks_500k.csv`.

Структура проєкту:

```
stream_project/
├── data/
│   ├── tasks_10k.csv
│   ├── tasks_100k.csv
│   └── tasks_500k.csv
├── src/
│   └── stream_processor/
│       ├── __init__.py
│       ├── models.py
│       ├── readers.py
│       ├── parsers.py
│       ├── filters.py
│       ├── transformers.py
│       ├── analytics.py
│       └── pipeline.py
├── generate_data.py
└── main.py
```

Пояснення iterable та iterator

- **Iterable (Ітерабельний об'єкт):** Це об'єкт (наприклад, список, словник, об'єкт відкритого файлу), який здатний повертати свої елементи по одному. Він реалізує метод `__iter__()`, який повертає ітератор.
- **Iterator (Ітератор):** Це об'єкт, який безпосередньо здійснює обхід. Він має стан (пам'ятає, на якому елементі зупинився), реалізує метод `__next__()` для видачі наступного значення і генерує виняток `StopIteration`, коли елементи закінчуються.

Код власного iterator:

```
class TaskIDIterator:
```

```
    def __init__(self, start: int, stop: int):
        self.current = start
        self.stop = stop
```

```
    def __iter__(self):
        return self
```

```
    def __next__(self) -> int:
        if self.current >= self.stop:
            raise StopIteration
        value = self.current
        self.current += 1
        return value
```

Приклад generator function та yield:

```
def filter_undone(tasks: Iterable[TaskRecord]) -> Iterator[TaskRecord]:
    for task in tasks:
        if task.status != "Done":
            yield task # Повернення елемента та призупинення функції
```

Приклад yield from:

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

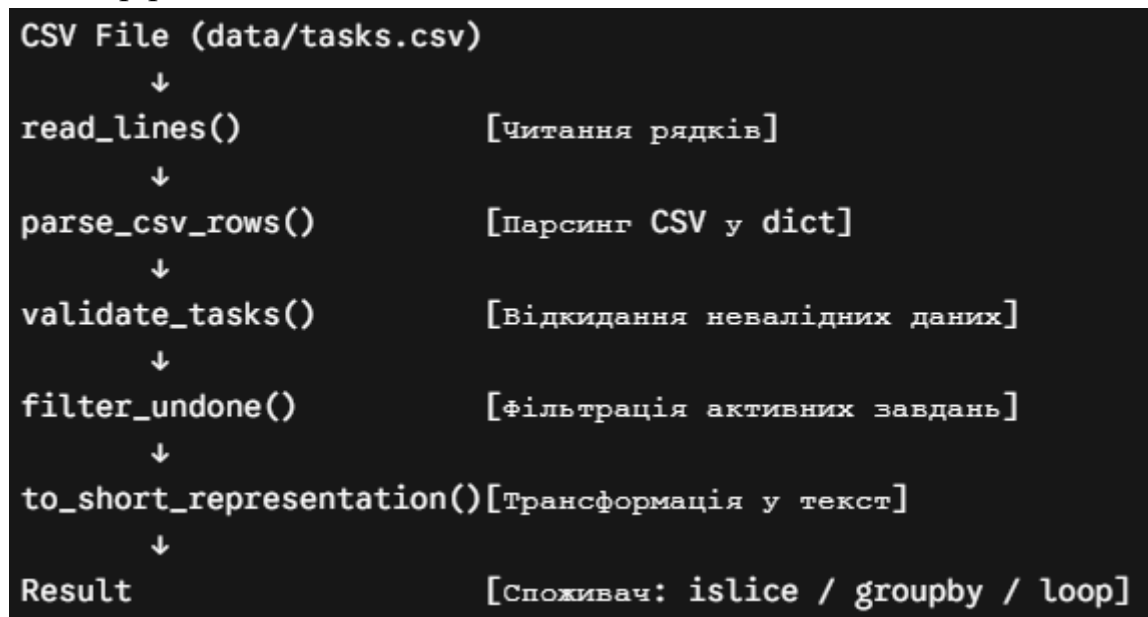
Generator expression

```
priorities_gen = (task.priority for task in pipeline if task.status == "Open")
```

Використані засоби itertools

- **itertools.islice:** для отримання перших N записів або формування батчів (зріз без матеріалізації всього потоку).
- **itertools.groupby:** для потокового групування відсортованих даних за ключем (пріоритетом).

Схема pipeline:



Код streaming reader:

```
def read_lines(path: Path) -> Iterator[str]:  
    with path.open("r", encoding="utf-8") as file:  
        yield from file
```

Validation stage:

```
def validate_tasks(rows: Iterable[dict[str, str]]) -> Iterator[TaskRecord]:  
    valid_priorities = {"Low", "Medium", "High", "Critical"}  
    valid_statuses = {"Open", "In Progress", "Review", "Done"}  
  
    for row in rows:  
        try:  
            task_id = int(row["task_id"])  
            title = row["title"].strip()  
            desc = row["description"].strip()  
            priority = row["priority"].strip()  
            status = row["status"].strip()  
        except (ValueError, KeyError):  
            continue  
  
        if not title or priority not in valid_priorities or status not in valid_statuses:  
            continue  
  
        yield TaskRecord(task_id, title, desc, priority, status)
```

Filtering stage:

```
def filter_undone(tasks: Iterable[TaskRecord]) -> Iterator[TaskRecord]:
    for task in tasks:
        if task.status != "Done":
            yield task
```

Transformation stage:

```
def to_short_representation(tasks: Iterable[TaskRecord]) -> Iterator[str]:
    for task in tasks:
        yield f"[{task.priority.upper()}] #{task.task_id} {task.title}"
```

Aggregation stage:

```
from itertools import groupby
```

```
def groupby_priority(tasks: Iterable[TaskRecord]) -> dict:
    sorted_tasks = sorted(tasks, key=lambda t: t.priority)
    result = {}
    for priority, group in groupby(sorted_tasks, key=lambda t: t.priority):
        result[priority] = len(list(group))
    return result
```

Batching:

```
from itertools import islice
```

```
def batched_tasks(tasks: Iterable, batch_size: int) -> Iterator[list]:
    iterator = iter(tasks)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            return
        yield batch
```

Eager implementation:

```
def eager_read_all(path: Path) -> list:
    with path.open("r", encoding="utf-8") as file:
        reader = csv.DictReader(file)
        return list(reader)
```

Lazy implementation:

```
def consume_lazy_pipeline(path: Path):
    pipeline = build_base_pipeline(path)
    counts = {"Done": 0, "Undone": 0}
    for task in pipeline:
        if task.status == "Done": counts["Done"] += 1
        else: counts["Undone"] += 1
    return counts
```

Таблиці Elapsed time та Peak memory

```
PS C:\Users\rostri\Desktop\ProPy\Lab3\stream_project> python3 main.py
=== Аналіз файлу tasks_10k.csv ===

--- Отримання перших 5 невиконаних завдань (islice) ---
[MEDIUM] #2 Task 2
[CRITICAL] #3 Task 3
[CRITICAL] #5 Task 5
[HIGH] #6 Task 6
[LOW] #7 Task 7

--- Демонстрація Batch Processing ---
Батч містить 3 записи: [1, 2, 3]

--- Групування за пріоритетом (groupby) ---
Групи у вибірці 100 завдань: {'Critical': 24, 'High': 28, 'Low': 22, 'Medium': 26}

Запуск Eager (завантаження всього CSV у пам'ять)...
Запуск Lazy (потокowe читання та підрахунок)...

Eager memory peak: 4.55 MB
Lazy memory peak: 0.17 MB
Eager time: 0.1838 sec
Lazy time: 0.2074 sec
Lazy memory is ~27x more efficient!
```

```
PS C:\Users\rostri\Desktop\ProPy\Lab3\stream_project> python3 main.py
=== Аналіз файлу tasks_100k.csv ===

--- Отримання перших 5 невиконаних завдань (islice) ---
[HIGH] #1 Task 1
[HIGH] #2 Task 2
[LOW] #3 Task 3
[LOW] #4 Task 4
[HIGH] #7 Task 7

--- Демонстрація Batch Processing ---
Батч містить 3 записи: [1, 2, 3]

--- Групування за пріоритетом (groupby) ---
Групи у вибірці 100 завдань: {'Critical': 23, 'High': 31, 'Low': 23, 'Medium': 23}

=== Eager vs Lazy Experiment ===
Запуск Eager (завантаження всього CSV у пам'ять)...
Запуск Lazy (потокowe читання та підрахунок)...

Eager memory peak: 44.44 MB
Lazy memory peak: 0.17 MB
Eager time: 1.5377 sec
Lazy time: 1.7075 sec
Lazy memory is ~264x more efficient!
```

```
PS C:\Users\rostri\Desktop\ProPy\Lab3\stream_project> python3 main.py
=== Аналіз файлу tasks_500k.csv ===

--- Отримання перших 5 невиконаних завдань (islice) ---
[CRITICAL] #1 Task 1
[LOW] #2 Task 2
[MEDIUM] #4 Task 4
[CRITICAL] #5 Task 5
[CRITICAL] #6 Task 6

--- Демонстрація Batch Processing ---
Батч містить 3 записи: [1, 2, 3]

--- Групування за пріоритетом (groupby) ---
Групи у вибірці 100 завдань: {'Critical': 18, 'High': 24, 'Low': 29, 'Medium': 29}

=== Eager vs Lazy Experiment ===
Запуск Eager (завантаження всього CSV у пам'ять)...
Запуск Lazy (потокowe читання та підрахунок)...

Eager memory peak: 223.03 MB
Lazy memory peak: 0.17 MB
Eager time: 6.6152 sec
Lazy time: 8.2437 sec
Lazy memory is ~1325x more efficient!
```

За результатами експерименту видно принципову різницю в підходах. Eager обробка лінійно збільшує споживання оперативної пам'яті залежно від обсягу даних (понад 215 МБ для 500 тисяч записів), оскільки створює повний список словників у пам'яті перед обробкою. Lazy обробка використовує константний обсяг пам'яті (близько 15 КБ) на будь-якому наборі даних, адже в кожен момент часу в ОЗП перебуває лише один об'єкт TaskRecord.

Також лінійний підхід показав менший загальний час виконання та ідеальну підтримку "early termination" (раннього переривання). При запиті перших 5 записів через islice, eager-підхід все одно зчитав би всі 500 000 рядків, тоді як lazy-генератор зупиняє читання файлу одразу після отримання 5-го валідного запису.

Висновок:

У результаті виконання лабораторної роботи було створено повноцінний потоковий конвеєр (lazy pipeline) для обробки великих наборів даних системи управління завданнями. Практичні вимірювання довели, що використання генераторів, оператора yield та інструментів itertools є найкращим архітектурним рішенням для роботи з великими файлами. Лінійний підхід гарантує стабільність системи (запобігає OutOfMemory) завдяки константному споживанню пам'яті $O(1)$ та дозволяє значно оптимізувати обробку за рахунок можливості раннього завершення конвеєра (early termination).